

Lecture 21 — Power Management and Energy-Aware Scheduling

Jeff Zarnett

2026-07-05

Limited Power

Motivating why we should care about power consumption is easy: the consumer market for buying laptops that last only an hour is tiny, if it exists at all. Similarly, you would not be very happy with a phone that doesn't make it through the average day. In fact, if your phone is not lasting long enough, you probably have decided you need either a new phone or, at least, a new battery. Even if we're not talking about running on battery, electricity has some cost to it. Energy is a resource, and something the operating system can manage (if it's aware of it). Operating systems can discover and configure power states through a standard interface called *ACPI*: Advanced Configuration and Power Interface. This has existed for decades now.

While it might be possible to manage power consumption across different devices in the system, given the limited time available to us in this course, we'll restrict the discussion to managing the power state of the CPU.

ACPI States. It's also important to differentiate this discussion from the general ACPI states of the device. The ACPI states are the ones the user typically thinks of: powered off, powered on, sleeping, and hibernating (off but state stored to disk to restore on wake-up).

If you close the lid of your laptop without selecting to shut down or power-off, it probably goes to a "sleep" mode. If it thinks that battery is getting low, it might transition to a "hibernate" mode by writing the contents of RAM to disk to restore when power comes back.

In this topic, we are considering the ACPI state to be that the computer is powered on.

Lay in a Course, Maximum Warp

There are certainly times where we want to use the CPU to its maximum power. If the laptop is plugged in to the charger and the user has a very CPU-intensive task running (play a game, render a video, execute a simulation, etc.) then it makes sense to go as hard as we can. Running the CPU at its maximum power means it delivers the best performance. But even in Star Trek, they don't go everywhere at Maximum Warp, do they?

You already know one reason why we should not run the CPU at 100% at all times from the title of this topic: energy consumption and therefore battery life. The other reason, of course, is heat. Heat can be uncomfortable if the laptop is in your lap or the phone in your hand, but heat is also the enemy of reliability. Running the components very hot will shorten their lifespan and leads to increased errors in calculations – but we'll come to that in the reliability topic.

Maybe a helpful way of looking at it: for the CPU to be at 100% is like running everywhere you go. Run to class, run to the lab, run to lunch... You *can* do it, sure, but depletes your energy way faster than walking and on a hot day you will definitely start feeling the negative effects of the heat. There's a time and place for it, but you don't run all the time. Instead, you adapt your pace of travel to the urgency of the situation and the state of resources. Want to go to class? Leisurely pace. Going to miss your bus? Jog for it, but not at max effort. Late for the exam? Maximum warp, Mr. Sulu.

The CPU has what are referred to as P-States and C-States. P is for "performance" and C is for "CPU idle". Broadly, P-States control the voltage and CPU frequency and C-States determine what the CPU does to reduce power consumption when no code is running (idle). For this section on P- and C-states, we'll dive into this helpful

guide [Bal18] that builds on some Intel Xeon data sheets, developer guides, and some kernel sources. Other CPUs with the same or other architectures also have comparable power management features, so the concepts here apply even if there are different names of things and different levels.

So, we have two levers available to reduce power consumption. Performance level and turning things off.

P-States. The first is the performance level setting, which says what clock speed we want to run the CPU at. This works because, to a first approximation, the power consumption of the CPU is linear with respect to frequency and quadratic with respect to voltage (the source writes it as $P \propto fV^2$ which is oversimplifying but directionally correct). A small amount of hardware knowledge helps in explaining why.

Your first-year electromagnetism course includes the formula $P = V \times I$ so increasing the voltage very obviously would increase the power consumption. The less-obvious component of this is that increasing the frequency is going to require a higher voltage to drive the components faster. This is, not coincidentally, why CPU clock speeds today are about the same as they were in 2005 (yes, Intel had a 3.8 GHz processor then!): going faster requires more voltage and more power and it quickly becomes a huge problem in terms of heat, power draw, error rates, and battery life. So CPU manufacturers have stopped chasing higher clock speeds.

P states are numbered: P0 is maximum frequency; any of the other number represent one of the lower frequency options offered by this hardware (the spec says a higher number is slower speed). The more modern multi-core CPUs permit having different power states for different cores, and thus there exists the idea of power states for the package (all cores) and for individual cores. But let's not get bogged down in that for now.

So, one way that we can immediately see improvement on power consumption is to set a higher-numbered P-state for the CPU and therefore run the CPU slower. Some devices do this automatically, e.g., a laptop moves to a lower-power state when unplugged from the charger. Or it can be user-initiated, when they choose the “low power” or “battery saver” mode. Some devices – including a previous phone I had – gave you the option of more than one battery-saver level which you can imagine corresponds to different P-state settings.

Now, on your phone (and probably other devices) it's more complicated than this, because going to low power mode changes more than the clock speed of the CPU. It also changes things like how often apps are permitted to refresh data in the background. We'll come back to that later in this topic.

Some CPUs also offer “boost” power where clock speed exceeds the nominal maximum for short periods of time. This only works when the chip is cool enough to do this without exceeding its heat limit, because the heat increases quite rapidly in this mode. This can, however, provide a meaningful performance boost that reduces latency if the workload is short but intense.

C-States. C-States are a lot simpler conceptually, because they are simply shutting down parts of the processor that are not currently being used. Akin to turning off the lights when you are not in a given room, or turning off wifi when you're out in the woods far away from any wifi access points, so that your phone doesn't waste any energy searching for a connection it will never find.

C-States are numbered, with C0 meaning everything is turned on and actively executing instructions. Any higher number than that means turning some things off. The ACPI standard 6.4¹ describes states C1, C2, and C3:

- C0: CPU is executing instructions and running normally.
- C1: CPU is halted (not executing instructions) but maintains cache and can resume very quickly if needed.
- C2: Lower-power than C1 and takes more time and higher transition energy cost to wake back up to C0 than from C1.

¹https://uefi.org/htmlspecs/ACPI_Spec_6_4.html/08_Processor_Configuration_and_Control/processor-power-states.html

- C3: Even lower-power than C2, caches not kept up to date, slowest time and highest energy cost to return to C0 state.

As with the P-states, modern multicore CPUs often have many more states than the minimum the spec requires, and differentiate between package-level and core-level C states. It's hard to do a real demo of this because it requires loading a kernel module, but we might be able to get an idea of it by looking at this CoreFreq GitHub project: <https://github.com/cyring/CoreFreq>.

If there is demand for the CPU, then the operating system may need to wake up a sleeping core (send it back to the C0 state). It can similarly send things to sleep with instructions. Hardware interrupts can also wake up a sleeping core. So when do we send cores to sleep? That's a really interesting topic.

Energy-Aware Scheduling

Energy-Aware Scheduling is, unsurprisingly, exactly what it sounds like: it's the scheduler making choices that reduce energy usage. We can break this down into a few different areas: sending cores to sleep & waking them up, choosing which cores are active at all, and managing background data & refreshes.

It's Bedtime. The OS can issue commands to the cores to send them to sleep; the idle task, for example, may contain the HLT (halt) instruction which puts it in C1. The OS can also use other instructions to send cores to other states above C1. The cost of waking up from C1 is negligible, but it's more expensive to come back from other states. So we would not send cores to this state unless we are reasonably sure that we won't be bringing it back for at least a little bit.

Putting a core to sleep (C1 or above) means the scheduler should not schedule threads to run on that core because it's not really ready to run threads right now. If demand for CPU is increasing and this seems like more than just a transient blip, it makes sense to re-activate some cores – starting first with the ones in C1 state and bringing up the other cores only if truly needed.

The specification makes clear that interrupts (the hardware kind) are a trigger to send a core back to the C0 state if it needs to be handled. More advanced CPUs can route hardware interrupts to core(s) that are already in the C0 state to reduce the impact on energy.

Big Cores and Little Cores. The kind of CPU organization in this course is and remains Symmetric Multiprocessing (SMP), but there's a little twist on this that is common in mobile devices: heterogeneous CPU topology, which is fancy words for saying we have cores of different size and power. The Linux kernel documentation about this topic² provides a good guide for how to handle this situation.

Hold On Now. User control is sometimes possible! In the settings of your phone you can choose whether apps are permitted to refresh in the background or if they can use cellular data (or only wifi). You could do this intentionally to save battery – but you probably don't want to limit it for everything. Turning it off for social media seems sensible – when you open the app it will show you whatever ads it was going to show you anyway – but you don't want to miss important notifications from messaging apps.

Background Killing. Mobile operating systems also kill background processes to stop them from running. This guarantees that the app isn't executing in the background and isn't using data. Since apps are rarely closed on the phone (yes, you can, but you have to go out of your way to do this), the OS is aggressive about this. You have probably experienced a symptom of this if you've been away from your mobile browser for a bit and when you switch to it, the page re-loads.

²<https://www.kernel.org/doc/html/v5.8/scheduler/sched-energy.html>

References

- [Bal18] Mete Balci. A minimum complete tutorial of cpu power management, c-states and p-states, 2018. Online; accessed 4-July-2026. URL: <https://metebalci.com/blog/a-minimum-complete-tutorial-of-cpu-power-management-c-states-and-p-states/>.